



**POLITÉCNICO
ESTELLA**

ASIGNATURA: DESARROLLO DE INTERFACES

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

DI06 TAREA

Presentado por: Eugen Moga

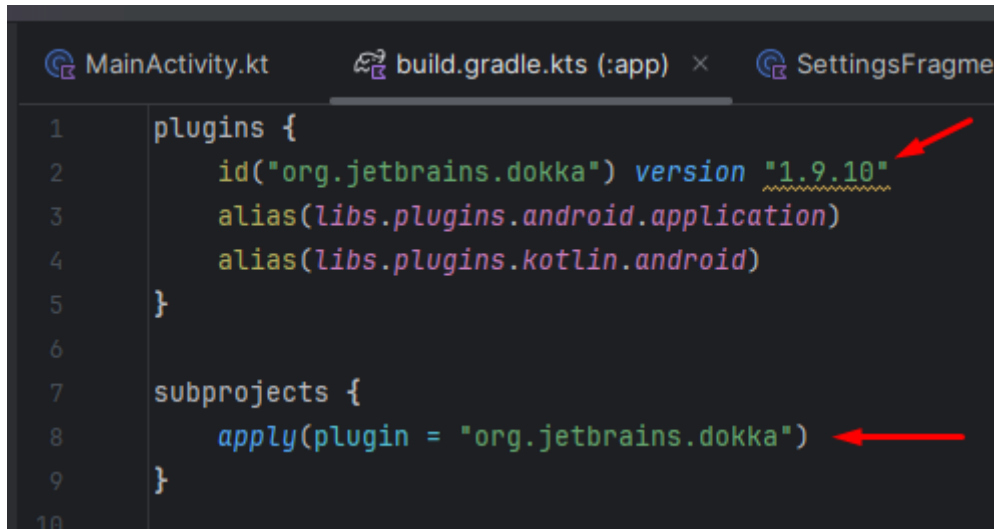
Fecha: 03/04/2026

ÍNDICE

Capturas de la documentación	1
Pagina HTML de Dokka	2
Clase MainActivity	3
Reflexión final	7

Capturas de la documentación

Documento las clases, luego configuro el plugin dokka en build.gradle para poder generar la documentación.

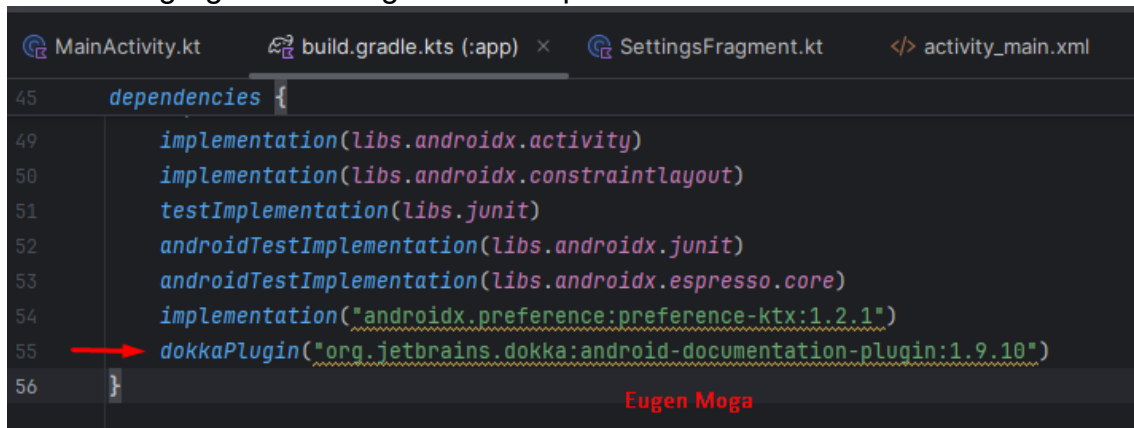


The screenshot shows the `build.gradle.kts` file for the `:app` module. It contains the following Kotlin code:

```
1 plugins {  
2     id("org.jetbrains.dokka") version "1.9.10"  
3     alias(libs.plugins.android.application)  
4     alias(libs.plugins.kotlin.android)  
5 }  
6  
7 subprojects {  
8     apply(plugin = "org.jetbrains.dokka")  
9 }  
10
```

Two red arrows point to the configuration: one to the version `"1.9.10"` on line 2, and another to the `apply(plugin = "org.jetbrains.dokka")` line on line 8.

También agrego dokkaPlugin en las dependencias.



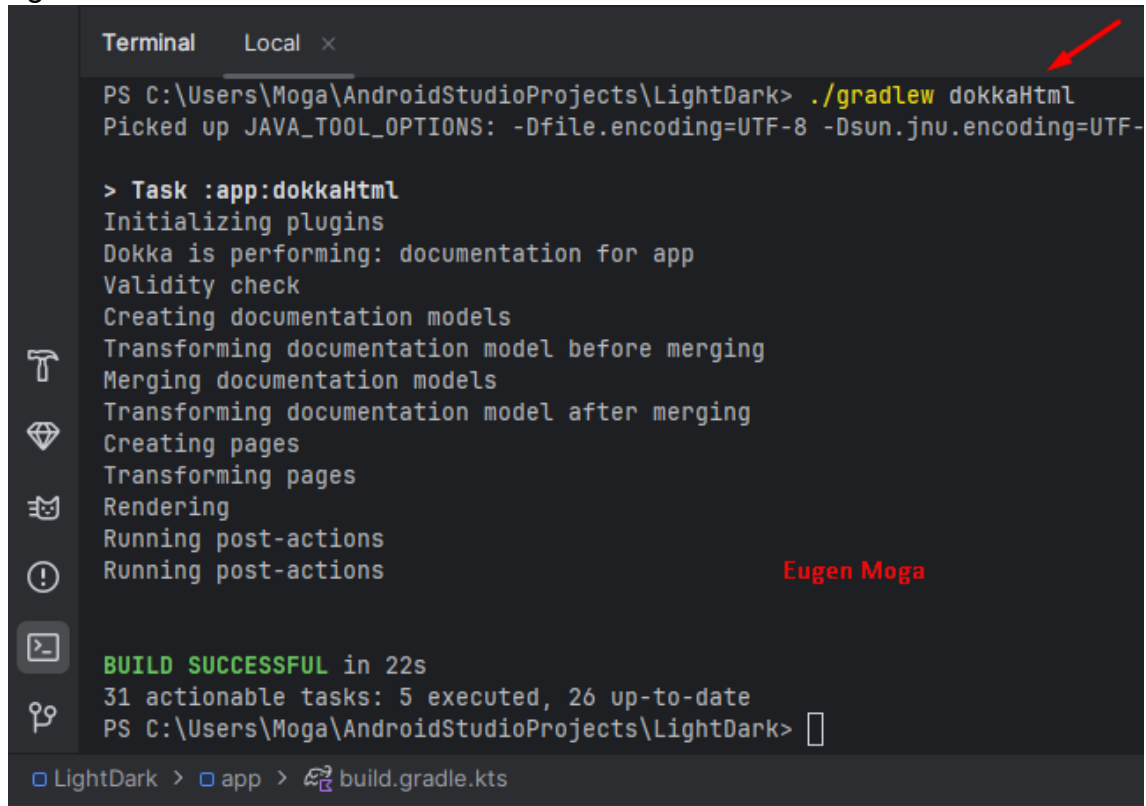
The screenshot shows the `dependencies` block in the `build.gradle.kts` file. It contains the following Kotlin code:

```
45 dependencies {  
49     implementation(libs.androidx.activity)  
50     implementation(libs.androidx.constraintlayout)  
51     testImplementation(libs.junit)  
52     androidTestImplementation(libs.androidx.junit)  
53     androidTestImplementation(libs.androidx.espresso.core)  
54     implementation("androidx.preference:preference-ktx:1.2.1")  
55     dokkaPlugin("org.jetbrains.dokka:android-documentation-plugin:1.9.10")  
56 }
```

A red arrow points to the `dokkaPlugin` line on line 55. The name `Eugen Moga` is visible in the bottom right corner of the editor.

Con los comentarios realizados y después de configurar el plugin dokka ya puedo ejecutar en la terminal de Android Studio el comando:

`./gradlew dokkaHtml`



```
Terminal Local x
PS C:\Users\Moga\AndroidStudioProjects\LightDark> ./gradlew dokkaHtml
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8

> Task :app:dokkaHtml
Initializing plugins
Dokka is performing: documentation for app
Validity check
Creating documentation models
Transforming documentation model before merging
Merging documentation models
Transforming documentation model after merging
Creating pages
Transforming pages
Rendering
Running post-actions
Running post-actions

BUILD SUCCESSFUL in 22s
31 actionable tasks: 5 executed, 26 up-to-date
PS C:\Users\Moga\AndroidStudioProjects\LightDark>
```

Este comando me acaba de generar la documentación del proyecto

Pagina HTML de Dokka

Para abrir la documentación voy a la carpeta:

`C:\Users\Moga\AndroidStudioProjects\LightDark\app\build\dokka\html`

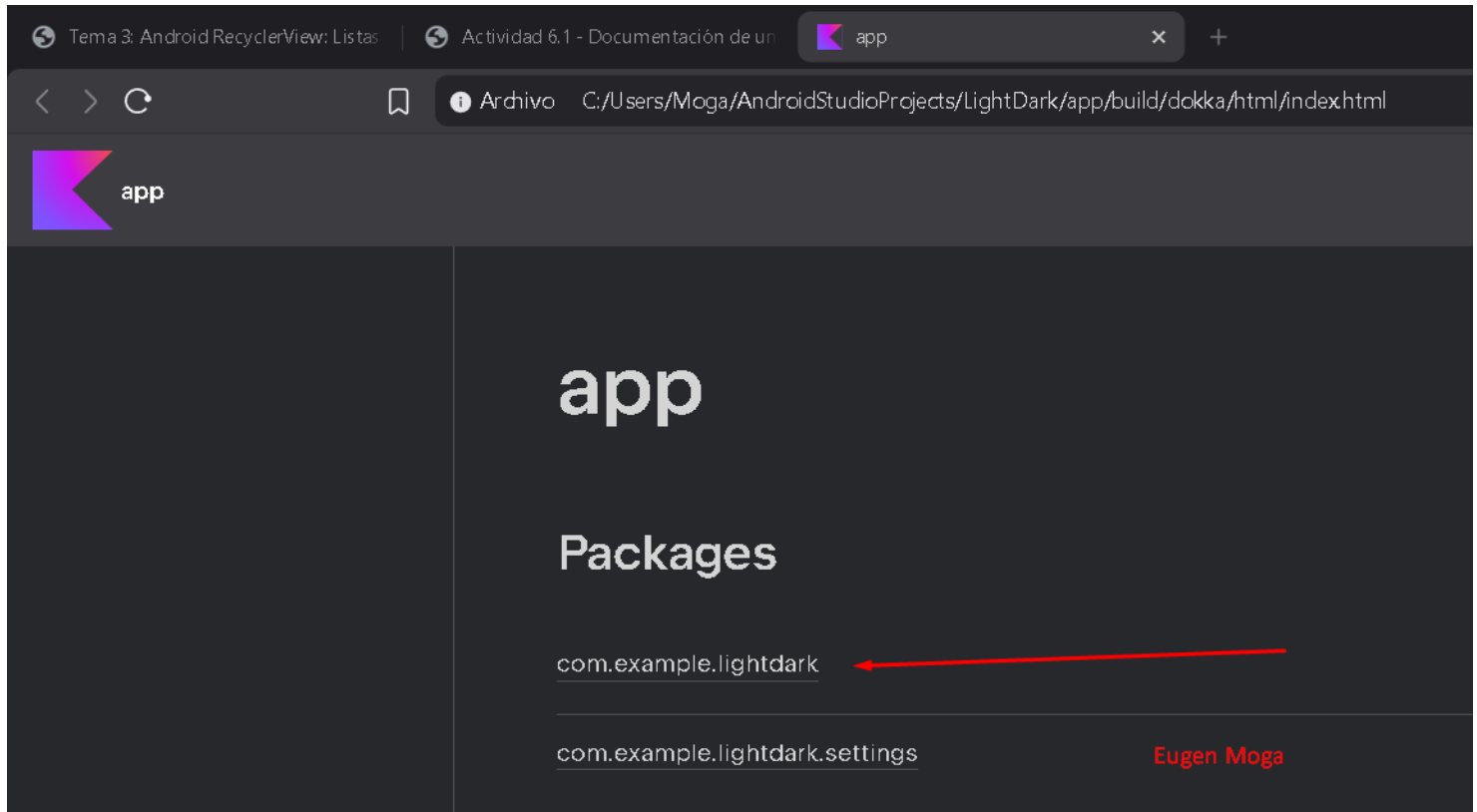
Y puedo ver el archivo `index.html` generado que contiene la documentación.



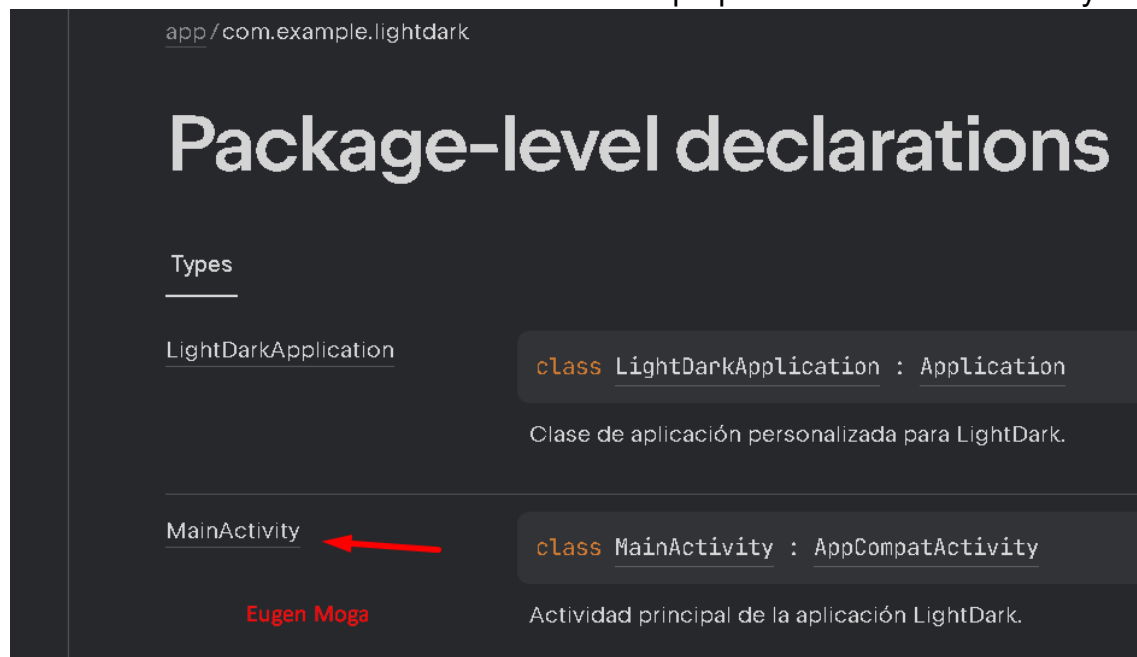
Nombre	Fecha de modificación
app	03/04/2026 8:50
images	03/04/2026 8:50
scripts	03/04/2026 8:50
styles	03/04/2026 8:50
index.html	03/04/2026 9:23
navigation.html	03/04/2026 9:23

Clase MainActivity

Al abrir el archivo index.html puedo ver los dos paquetes que contienen clases comentadas. Para ver la clase MainActivity abro el primer paquete `com.example.lightdark`



Puedo ver las dos clases comentadas de este paquete. Yo abro MainActivity



Nada mas entrar puedo ver el comentario principal

app/com.example.lightdark/MainActivity

MainActivity

```
class MainActivity : AppCompatActivity
```

Actividad principal de la aplicación LightDark.

Gestiona la pantalla de inicio con un formulario de email, una barra de herramientas superior y una barra inferior con acciones. También permite la navegación hacia los ajustes de tema (claro/oscuro).

Author
Eugen Moga

Members

El siguiente comentario define la función onCreate pero el comentario que configure no se muestra porque es una función heredada de la clase AppCompatActivity

onCreate

```
open fun onCreate(p0: Bundle?, p1: PersistableBundle?)
```

Pasa lo mismo con todas las funciones que heredan y sobrescriben el método de otra clase como: onCreateOptionsMenu, onOptionsItemSelected.

Por último, en la clase MainActivity tengo la función validateEmailField, a esta función se llamaba validate y le cambie el nombre para que sea más descriptivo.

[validateEmailField](#) 

```
fun validateEmailField(view: View)
```

Valida el campo de email del formulario. Muestra un mensaje de error si el campo está vacío, o lo limpia si contiene algún valor.

Eugen Moga

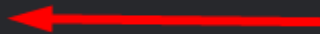
Si pincho en la función también puedo ver el parámetro view definido en el comentario. Si tuviera más parámetros también se verían definidos.

validateEmailField

```
fun validateEmailField(view: View)
```

Valida el campo de email del formulario. Muestra un mensaje de error si el campo está vacío, o lo limpia si contiene algún valor.

Parameters



view

Eugen Moga

La vista que dispara la acción (botón de validar).

La función que cambia el tema es `applyTheme` y esta en la clase `ThemeSetup.kt` para encontrarla en la documentación tengo que abrir el paquete `com.example.lightdark.settings`

applyTheme

```
fun applyTheme(selectedTheme: String?, context: Context)
```

Aplica el tema de la aplicación según el valor de preferencia recibido.

Parameters

<code>selectedTheme</code>	Nombre del tema seleccionado: <code>DARK_THEME</code> , <code>LIGHT_THEME</code> o cualquier otro valor para usar el tema del sistema.
<code>context</code>	Contexto de la aplicación o actividad.

```
fun applyTheme(context: Context)
```

Lee la preferencia de tema almacenada y aplica el modo correspondiente.

Usa `PreferenceManager.getDefaultSharedPreferences` para obtener el valor guardado bajo la clave `"theme"`. Si no existe ninguna preferencia, aplica el tema `DEFAULT_THEME` (sigue al sistema).

Parameters

<code>context</code>	Contexto de la aplicación o actividad desde donde se llama.
----------------------	---

Eugen Moga

En cuanto a las variables relacionadas con el modo claro y oscuro yo no las tenía configuradas, se lo pasaba hardcoded a una condición `if`. Esto lo he modificado y he agregado las variables correspondientes. Y en vez de utilizar un `if` lo hago con un bucle `when` y pasándole las variables. Por tanto estas son las variables que he implementado.

Properties

`DARK_THEME`

```
const val DARK_THEME: String
```

Valor de preferencia que activa el modo oscuro.

`DEFAULT_THEME`

```
const val DEFAULT_THEME: String
```

Valor de preferencia que sigue el tema del sistema.

`LIGHT_THEME`

```
const val LIGHT_THEME: String
```

Eugen Moga

Valor de preferencia que activa el modo claro.

Reflexión final

¿Para qué sirve documentar código? ¿Qué parte te ha resultado más difícil?

Documentar el código es muy útil para explicar el funcionamiento del programa a la hora de programar. Y a su vez permite que otros desarrolladores entiendan el funcionamiento que tiene cada clase y función.

Lo que me parece más difícil es entender que tengo que poner en los `@param` y `@return` de las funciones heredadas de otras clases como `onCreate`, `onCreateOptionsMenu` o `onOptionsItemSelected`. Además como son funciones heredadas que sobrescriben(`override`) las funciones ya definidas en la documentación no se reflejan mis comentarios.

Otra cosa que me parece difícil de entender en la documentación es que en la clase `MainActivity` aparecen cientos de métodos que no estoy utilizando en la clase. Y esto es debido a que `MainActivity` hereda de `AppCompatActivity` que a su vez hereda de otras muchas clases. Eso hace bastante lioso de entender el código al menos para mí.

¿Crees que es útil en proyectos reales?

Sí creo que es muy útil, especialmente en proyectos con varios desarrolladores o de larga duración. Sin documentación, el código heredado es muy difícil de mantener. En proyectos reales es prácticamente obligatorio.